



SECURE FILE TRANSFER MONITORING SYSTEM

SFTMS — Complete Project Report

Domain: Cybersecurity / System Administration

Stack: Python · Watchdog · SHA-256 · SMTP · Streamlit · OpenPyXL

Prepared by: Aryan Gupta | Date: 27th May 2026

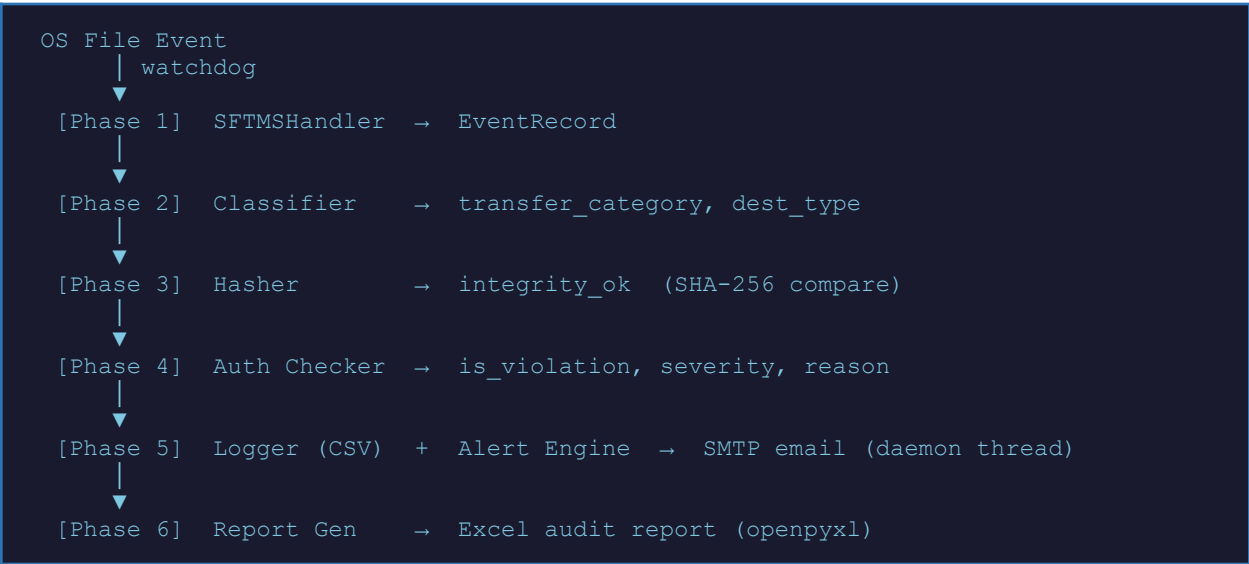
1. Project Overview

Objective	Real-time, silent monitoring of file system directories for unauthorised transfers, integrity violations, and exfiltration attempts
Core Language	Python 3.x
Key Libraries	watchdog, pandas, hashlib, smtplib, streamlit, openpyxl
Architecture	6-phase modular pipeline: Watch → Classify → Hash → Auth → Log → Alert
Deployment	Local daemon (Linux/Windows); Dashboard via Streamlit Cloud
Config Source	Environment variables — no secrets hardcoded

SFTMS is a lightweight background daemon that watches any directory for file system events — creations, modifications, moves, and deletions — and runs each event through a five-stage security pipeline in real-time. The sixth stage generates an on-demand Excel audit report for compliance reviews.

2. System Architecture & Pipeline

The system uses a modular pipeline design. Each stage enriches a single EventRecord object and passes it to the next stage. No stage has side-effects on another stage's state.



3. Phase Breakdown

Phase	Name	What it does	File
Phase 1	Foundation	Watchdog listener, EventRecord dataclass	watcher.py
Phase 2	Classification	CSV logging, event categorisation	classifier.py
Phase 3	Integrity Hashing	SHA-256 per-file baseline compare	hasher.py
Phase 4	Auth & Detection	Whitelist, rate limiting, violation flags	auth_checker.py
Phase 5	Alert Engine	SMTP email, daemon thread, cooldown	alert.py
Phase 6	Audit Reporting	Multi-sheet Excel report (openpyxl)	report_generator.py

Phase 1 — Foundation (watcher.py)

- Uses the watchdog library to hook into OS-level inotify / FSEvents / ReadDirectoryChangesW APIs.
- The SFTMSHandler class overrides on_created, on_modified, on_moved, on_deleted.
- Filters out temp/OS artefacts (.tmp, .swp, .DS_Store) before building any record.
- Builds an EventRecord dataclass with: timestamp, event_type, src/dest paths, username, file size.

Phase 2 — Classification (classifier.py)

- Maps raw event types to transfer categories: created→copy_in, modified→edit, moved→transfer, deleted→removal.
- Classifies destination type (usb / cloud / temp / network / local / n/a) via path fragment matching.
- Thread-safe CSV appends using a threading.Lock — safe for concurrent watchdog callbacks.

Phase 3 — Integrity Hashing (hasher.py)

- Builds a baseline_hashes.json on first run by scanning WATCH_PATH and computing SHA-256 for every file.
- On each modified event: recomputes hash and compares to baseline. Mismatch sets integrity_ok = False.
- Handles created (add to baseline), moved (rename key), deleted (remove key) cleanly.
- Reads files in 64 KB chunks — handles large files without loading them fully into RAM.

Phase 4 — Auth Check & Violation Flagging (auth_checker.py)

Four independent checks run on every event. The highest-severity finding wins:

Severity	Trigger
LOW	File moved to unrecognised / unknown destination
MEDIUM	Unlisted user performing a file transfer
HIGH	Transfer to sensitive dest (USB / cloud / /tmp) or rate limit breach
CRITICAL	SHA-256 hash mismatch — file tampered after baseline was set

- Rate limiting uses a per-user collections.deque. Entries older than RATE_WINDOW_SECONDS are pruned on each event.
- All reasons from multiple checks are joined with ' | ' and preserved in the violation_reason field.

Phase 5 — Alert Engine (alert.py)

- Only violations at or above ALERT_SEVERITY_THRESHOLD (default: MEDIUM) trigger an email.
- Cooldown mechanism: a (username, severity) dict prevents re-alerting the same user within ALERT_COOLDOWN_SECONDS.
- SMTP runs in a daemon thread — the watchdog observer loop is never blocked by network latency.
- Supports both SSL (port 465) and STARTTLS (port 587) automatically based on configured port.
- All credentials come from environment variables; nothing is hardcoded.

Phase 6 — Audit Report Generation (report_generator.py)

Generates a multi-sheet Excel workbook on demand via python main.py --report.

Sheet	Contents
Summary	Total events, events by type, total violations, violations by severity, integrity failure count
Top Users	Top 10 most active usernames and their event counts
Hourly Activity	Event distribution across 24 hours (useful for detecting off-hours activity)
Full Log	Complete transfer_log.csv reproduced as a formatted sheet
Violations	Filtered violations only (present only when violations exist)

Corrected generate_report() — Key Sections

```
def generate_report():
    # Fix #3 — mkdir guard so standalone run works
    os.makedirs(settings.REPORTS_DIR, exist_ok=True)

    ...

    # Fix #1 — boolean string comparison
    if 'integrity_ok' in log_df:
        integrity_failures = len(
            log_df[log_df['integrity_ok'].astype(str).str.strip().str.lower() == 'false']
        )

    # Fix #4 — reassign instead of inplace
    log_df = log_df.drop(columns=['timestamp_dt'])
```

config/settings.py — REPORTS_DIR Fix (Bug #2)

```
# Before (mixes source code with data artefacts):
REPORTS_DIR = str(BASE_DIR / 'reports')

# After:
REPORTS_DIR = str(BASE_DIR / 'data' / 'reports')
```

5. Configuration Reference

Setting	Default	Purpose
WATCH_PATH	./test_watch	Directory to monitor
WHITELIST_USERS	admin, security_team	Users exempt from MEDIUM violations
SENSITIVE_DESTINATION S	/media/, /tmp/, dropbox...	Path fragments that trigger HIGH alerts
MAX_FILES_PER_WINDOW	20	Rate limit — max events per user per window
RATE_WINDOW_SECONDS	60	Seconds in the rate-limit sliding window
ALERT_SEVERITY_THRESHOLD	MEDIUM	Minimum severity to send an email alert
ALERT_COOLDOWN_SECONDS	60	Suppress re-alerts for same user/severity
SFTMS_SMTP_HOST	smtp.gmail.com	SMTP server hostname (env var)
SFTMS_SMTP_PORT	465	465 = SSL, 587 = STARTTLS (env var)

6. The 60-Second Manager Pitch (Screen-Share Mode)

Use this script when screen-sharing. Each row is a timed segment. Have two windows pre-open before the meeting: terminal in WATCH_PATH directory, and browser on localhost:8501 (Streamlit).

Time	Focus	What to say / show
0–10 s	Hook / Problem	Pull up terminal or violations.csv on screen. Say: "We had a gap — no visibility into who moves sensitive files or whether they've been tampered with. This tool closes that gap."
10–25 s	Live Demo	Open test_watch/, modify confidential.docx.txt. Terminal fires: [MODIFIED] ... [CRITICAL VIOLATION] hash mismatch. Point to severity label. "That alert just fired because the file's hash changed — possible tampering."
25–40 s	Dashboard	Switch to browser (streamlit run app.py). Show the bar charts: events by category, violations by severity. "All of this is logged automatically — no manual effort."
40–55 s	Value & Scope	"It monitors any directory, whitelists approved users, rate-limits bulk transfers, and emails the security team instantly. Zero external dependencies beyond Python."
55–60 s	Close	"Phase 6 gives us an Excel audit report on demand. Ready to point it at a real directory whenever you'd like."

What to Have Open Before the Meeting

- Terminal 1 — python main.py running (monitor active, waiting for events)
- Terminal 2 — cd test_watch/ ready to edit a file
- Browser — http://localhost:8501 with the Streamlit dashboard open
- violations.csv open in VS Code or Excel as a backup talking point

3 Phrases That Land Well With Managers

Say this	Because it shows
"Zero polling — the OS tells us instantly."	You understand the architecture, not just that it works.
"The alert fires on a background thread so the monitor never misses an event."	You thought about reliability and race conditions.
"Credentials come from environment variables — nothing hardcoded."	You understand production security practices.

7. Quick-Start Commands

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Create test environment
python main.py --setup

# 3. Start the live monitor
python main.py

# 4. Launch the dashboard (separate terminal)
streamlit run app.py

# 5. Generate Excel audit report
python main.py --report

# SMTP quick-start (Gmail)
export SFTMS_SMTP_HOST=smtp.gmail.com
export SFTMS_SMTP_PORT=465
export SFTMS_SMTP_USER=you@gmail.com
export SFTMS_SMTP_PASSWORD=your_app_password
export SFTMS_ALERT_RECIPIENT=security@yourorg.com
```